

tf.py_function Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/py_function

Wraps a python function into a TensorFlow op that executes it eagerly.

Function Signatures

```
tf.py_function( func=None, inp=None, Tout=None, name=None )
```

Code Examples

```
tf.py_function(  
    func=None, inp=None, Tout=None, name=None  
)
```

```
tf.py_function(  
    func=None, inp=None, Tout=None, name=None  
)
```

```
@tf.py_function(Tout=tf.float32)  
def py_log_huber(x, m):  
    print('Running with eager execution.')  
    if tf.abs(x) <= m:  
        return x**2  
    else:  
        return m**2 * (1 - 2 * tf.math.log(m) + tf.math.log(x**2))
```

```
@tf.py_function(Tout=tf.float32)
```

```
def py_log_huber(x, m):
```

```
    print('Running with eager execution.')
```

```
    if tf.abs(x) <= m:
```

```
        return x**2
```

Documentation

Wraps a python function into a TensorFlow op that executes it eagerly.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.py_function`

Used in the notebooks

- Better performance with the `tf.data` API
 - `tf.data`: Build TensorFlow input pipelines
 - Better performance with `tf.function`
 - `TfRecord` and `tf.train.Example`

Using `tf.py_function` inside a `tf.function` allows you to run a python function using eager execution, inside the `tf.function`'s graph.

This has two main effects:

- This allows you to use `nofunc=None`, `inp=None`, `Tout=None` tensorflow code

inside your `tf.function`.

- It allows you to run python control logic in a `tf.function` without

relying on `tf.autograph` to convert the code to use tensorflow control logic (`tf.cond`, `tf.while_loop`).

Both of these features can be useful for debugging.

Since `tf.py_function` operates on Tensors it is still differentiable (once).

There are two ways to use this function:

As a decorator

Use `tf.py_function` as a decorator to ensure the function always runs eagerly.

When using `tf.py_function` as a decorator:

- you must set `Tout`
- you may set `name`
- you must not set `func` or `inp`

For example, you might use `tf.py_function` to implement the log huber function.

Under eager execution the function operates normally:

Inside a `tf.function` the `tf.py_function` is not converted to a `tf.Graph`:

The `tf.py_function` only executes eagerly, and only when the `tf.function` is called:

Gradients work as expected:

Inplace

You can also skip the decorator and use `tf.py_function` in-place.

This form is a useful shortcut if you don't control the function's source, but it is harder to read.

More info

You can also use `tf.py_function` to debug your models at runtime using Python tools, i.e., you can isolate portions of your code that you want to debug, wrap them in Python functions and insert `pdb` tracepoints or print statements as desired, and wrap those functions in `tf.py_function`.

For more information on eager execution, see the [Eager guide](#).

`tf.py_function` is similar in spirit to `tf.numpy_function`, but unlike the latter, the former lets you use TensorFlow operations in the wrapped Python function. In particular, while `tf.compat.v1.py_func` only runs on CPUs and wraps functions that take NumPy arrays as inputs and return NumPy arrays as outputs, `tf.py_function` can be placed on GPUs and wraps functions that take Tensors as inputs, execute TensorFlow operations in their bodies, and return Tensors as outputs.

- Calling `tf.py_function` will acquire the Python Global Interpreter Lock

(GIL) that allows only one thread to run at any point in time. This will preclude efficient parallelization and distribution of the execution of the program.

- The body of the function (i.e. `func`) will not be serialized in a

GraphDef. Therefore, you should not use this function if you need to serialize your model and restore it in a different environment.

- The operation must run in the same address space as the Python program

that calls `tf.py_function()`. If you are using distributed TensorFlow, you must run a `tf.distribute.Server` in the same process as the program that calls `tf.py_function()` and you must pin the created operation to a device in that server (e.g. using with `tf.device():`).

- Currently `tf.py_function` is not compatible with XLA. Calling

`tf.py_function` inside `tf.function(jit_compile=True)` will raise an error.

Calling `tf.py_function` will acquire the Python Global Interpreter Lock (GIL) that allows only one thread to run at any point in time. This will preclude efficient parallelization and distribution of the execution of the program.

The body of the function (i.e. `func`) will not be serialized in a GraphDef. Therefore, you should not use this function if you need to serialize your model and restore it in a different environment.

The operation must run in the same address space as the Python program that calls `tf.py_function()`. If you are using distributed TensorFlow, you must run a `tf.distribute.Server` in the same process as the program that calls `tf.py_function()` and you must pin the created operation to a device in that server (e.g. using with `tf.device():`).

Currently `tf.py_function` is not compatible with XLA. Calling

`tf.py_function` inside `tf.function(jit_compile=True)` will raise an error.

- If `func` returns a `Tensor` (or a value that can be converted to a

`Tensor`): the `tf.DType` for that value. * If `func` returns a

`CompositeTensor`: The `tf.TypeSpec` for that value. * If `func` returns

`None`: the empty list (`[]`). * If `func` returns a list of `Tensor` and

`CompositeTensor` values: a corresponding list of `tf.DTypes` and `tf.TypeSpecs` for each value.

Returns

- If `func` is `None` this returns a decorator that will ensure the decorated function will always run with eager execution even if called from a `tf.function/tf.Graph`.

- If used `func` is not `None` this executes `func` with eager execution

and returns the result: a `Tensor`, `CompositeTensor`, or list of `Tensor` and `CompositeTensor`; or an empty list if `func` returns `None`.